

Web Semantics Project: Knowledge Graph Construction and Embedding

Myriam & Harrisson

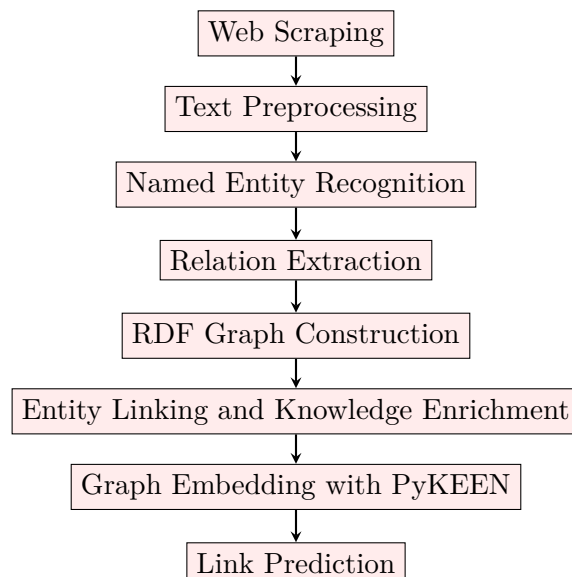
April 7, 2025

Contents

1	Part 1 : Knowledge Graph Construction and Web Scraping	2
1.1	Text Preprocessing	2
1.2	Named Entity Recognition (NER)	3
1.3	Comparison and Evaluation: spaCy vs CRF	3
1.4	Relation Extraction	3
1.5	RDF Graph Construction	4
1.6	Web Scraping	4
2	Part 2 : Graph Embedding with PyKEEN	5
2.1	Entity Linking and Knowledge Enrichment	5
2.2	Disambiguation Strategy	6
2.3	Embedding Structure Analysis	7
2.4	Model Training and Evaluation: Link Prediction	7
2.5	Initial Results and Fine-Tuning	8
2.6	Comparative Analysis of Embedding Models	8
3	Visualizations and Examples	8
3.1	Entity Embedding Projections (PCA and t-SNE)	9
3.2	Relation Embedding Projections (PCA and t-SNE)	10
3.3	Insights.	12
4	References Used	12
5	Conclusion	13

Abstract

This report presents the methodology and results of constructing a knowledge graph from English news articles. It details all steps from data collection, entity and relation extraction, RDF structuring, entity enrichment with DBpedia/Wikidata, to embedding using PyKEEN. The report highlights practical challenges and provides a full reproducible pipeline.



Overview of the Knowledge Graph Construction and Embedding Pipeline

1 Part 1 : Knowledge Graph Construction and Web Scraping

This project aimed to build a knowledge graph from news articles, covering all steps from data collection to RDF structuring and vector embedding. It deepened our skills in information extraction, NLP, and semantic modeling, while reinforcing a methodical approach to technical problem-solving. This report outlines our process, choices, challenges, and solutions, with examples and visualizations.

1.1 Text Preprocessing

Before applying NER models, we implemented a complete text preprocessing pipeline. This included:

- Lowercasing all tokens
- Removing stop words using NLTK's English list
- Removing most punctuation, but keeping hyphens
- Lemmatizing tokens using `en_core_web_trf`

The function `preprocess_text()` was applied to both synthetic and real-world sentences. We also processed the full CoNLL dataset using batch execution via `nlp.pipe()` to improve performance.

Example result:

```
"The US accepts a French chat to welcome english lamb."  
→ [us, accept, french, chat, welcome, english, lamb]
```

1.2 Named Entity Recognition (NER)

Feature Engineering for CRF. We implemented a Conditional Random Field (CRF) model using `sklearn-crfsuite`, relying on token-level and contextual features:

- `word.lower()`: lowercase form
- `word.istitle()`, `word.isdigit()`: orthographic features
- `prev_word.lower()`, `next_word.lower()`: context window
- Sentence boundary markers: BOS, EOS

These features help the model capture context and identify named entity patterns (e.g., sequences of capitalized words).

1.3 Comparison and Evaluation: spaCy vs CRF

To evaluate our NER pipeline, we compared two approaches:

- **CRF** — a supervised model trained on CoNLL-2003 using hand-crafted features.
- **spaCy** — a pretrained transformer model `en_core_web_trf`, chosen over `WebSM` due to Jupyter compatibility issues.

Both models were tested on the same annotated dataset with entity types PER, LOC, ORG, and MISC, using precision, recall, and F1-score as metrics.

Per-Entity Example Scores:

- **CRF**:
 - B-LOC: 0.86 precision, 0.80 recall, 0.83 F1
 - I-PER: 0.98 precision, 0.99 recall, 0.98 F1
- **spaCy** (with `evaluate_spacy()`):
 - B-ORG: 0.78 precision, 0.11 recall, 0.19 F1

Overall Performance Comparison:

Model	Precision	Recall	F1-score
CRF	0.952	0.953	0.952
spaCy	0.739	0.834	0.772

Interpretation Summary:

- **CRF** achieved excellent scores due to training on the same BIO scheme and the ability to learn token-level dependencies (e.g., B-PER $\rightarrow$ I-PER), which aligns well with the structure of news articles.
- **spaCy**, although powerful in general contexts, showed limitations in this task. The model underperformed on multi-token entities due to imperfect alignment and lack of task-specific fine-tuning.

1.4 Relation Extraction

We used a hybrid strategy combining syntactic and regex-based patterns to extract relations.

1. Dependency-based patterns: Using spaCy’s syntactic dependencies, we extracted relations based on rules like:

- (nsubj) + verb + (pobj)
- (nsubjpass) + verb + agent

2. Regex-based semantic rules: We designed handcrafted patterns for relations such as:

- `hasSibling`: “Barack has a sibling named Michelle.”
- `isMarriedTo`, `hasAward`, `hasEducation`, `worksAt`, `hasChild`, `founded`, etc.

We mapped each relation to DBpedia predicates via `relation_map` (e.g., `hasAward` $\rightarrow$ `dbp:award`). This ensured semantic compatibility and improved the interpretability of the final RDF graph.

1.5 RDF Graph Construction

We used `RDFlib` to create the knowledge graph:

- Defined custom namespaces
- Created triples using `URIRef` and `Literal`
- Serialized the graph in RDF/XML format

SPARQL Validation Queries: To verify consistency, we executed queries such as:

- `SELECT ?entity ?type` to retrieve all entity types
- `SELECT ?entity WHERE { ?entity rdf:type PERSON }` to list all persons

The RDF graph contained several hundred valid and semantically coherent triples.

1.6 Web Scraping

Politico (Attempted). We initially designed a full-featured web scraping pipeline for `politico.com`, including cookie dismissal, modal handling, and recursive link extraction using Selenium and BeautifulSoup. However, this pipeline was ultimately abandoned due to repeated blocking by the site’s anti-bot mechanisms. These included:

- CAPTCHA challenges and dynamic redirections/ IP bans after a few requests,
- Consent and subscription walls triggered during crawling/ Inconsistent page structures breaking scraping logic.

We attempted various workarounds (random delays, modified headers, and modal removal using JavaScript), but the results remained unstable and difficult to reproduce. As a result, the Politico pipeline was kept in the codebase for documentation purposes but was not used in the final knowledge graph.

NPR News (Final Source). Due to the limitations encountered with Politico, we shifted to a more stable source: `npr.org`. This switch proved more effective, enabling us to scrape news content with higher reliability. We developed a lightweight but robust scraping setup using `requests`, `BeautifulSoup`, and fallback mechanisms.

Our NPR scraping module performed the following:

- Sent GET requests to the `/sections/world/` page and extracted the top article links.
- For each article, downloaded the HTML and parsed the title, URL, and body text.

- Used multiple selector strategies to adapt to layout variations (e.g., `article p, div[class*="article"] p`).
- Removed irrelevant content such as captions, promotional text, and empty lines.
- Stored valid articles in structured JSON format, including:
 - `title` (string)
 - `url` (string)
 - `content` (cleaned text body)

This process yielded 24 high-quality English articles, which were later processed for entity and relation extraction. The data served as a real-world testbed for validating our knowledge graph pipeline.

2 Part 2 : Graph Embedding with PyKEEN

In this second stage, we transitioned from symbolic representation (triples in RDF) to dense semantic embedding using PyKEEN. This allowed us to perform deeper structural analysis of the graph and enabled advanced downstream tasks such as link prediction.

2.1 Entity Linking and Knowledge Enrichment

To enrich our initial RDF graph with external verified knowledge, we implemented a lightweight enrichment pipeline. The approach focused on **additive augmentation** without attempting full disambiguation or canonicalization.

For each named entity (subject or object) extracted from the RDF graph, we queried DBpedia using an **exact label match** to retrieve related facts (predicate-object pairs) for the corresponding resource.

Pipeline Summary:

1. Extract label from the URI (e.g., `http://example.org/Angela_Merkel` → “Angela Merkel”)
2. Query DBpedia via `rdfs:label` to fetch up to 100 associated facts
3. Add the new triples to the graph using the original URI (no replacement or rewriting)

This enrichment is **additive**:

- **No URI substitution** or `owl:sameAs` declarations
- New triples are directly linked to original URIs
- Preserves semantic consistency while increasing graph density

Example Enrichment: Given the triple:

(Angela Merkel, wasElectedAs, Chancellor)

DBpedia enrichment added the following facts:

- `ex:Angela_Merkel dbp:birthPlace dbr:Hamburg`
- `ex:Angela_Merkel dbp:spouse dbr:Joachim_Sauer`

Final Statistics. Before enrichment, the RDF graph contained **78 triples** and **119 unique entities**. After enrichment, the graph expanded to **4233 triples** and **2812 unique entities**. We enriched the first 100 extracted entities to avoid API rate limits and ensure stability.

2.2 Disambiguation Strategy

Disambiguating entities is essential to ensure semantic accuracy and avoid incorrect merges within the knowledge graph.

Initial Approach (Discarded). We first explored a multi-step pipeline combining:

- Text normalization,
- DBpedia Spotlight annotation,
- Fallback strategies using Wikidata and fuzzy string matching.

However, this approach suffered from:

- Inconsistencies in entity resolution,
- Limited recall due to sparse context,
- Significant performance issues.

Final Strategy (Kept). To balance simplicity and robustness, we adopted a streamlined method:

- Extract surface forms from RDF URIs,
- Perform exact label lookups on DBpedia using SPARQL,
- Append enriched triples to the existing graph (without modifying URIs).

This approach allowed us to:

- Expand the graph semantically,
- Avoid premature or incorrect disambiguation,
- Support downstream tasks like embedding and link prediction.

Limitations and Future Work.

- No caching, deduplication, or confidence scoring was implemented.
- Entities without a DBpedia match were skipped.
- Future improvements may include:
 - Contextual disambiguation strategies,
 - Confidence thresholds for candidate matches,
 - Canonical URI usage (e.g., with `owl:sameAs`) to improve interoperability.

Processing Step	Count
Total Named Entities Extracted	119
Entities Sent for DBpedia Enrichment	100
Entities Successfully Enriched	100
Total RDF Triples Before Enrichment	78
Total RDF Triples After Enrichment	4233
Unique Entities in Final Graph	2812

Total number of RDF triples: 78
Total number of unique entities: 119

Figure 1: Before enrichment

Total number of RDF triples after enrichment: 4233
Total number of unique entities after enrichment: 2812

Figure 2: After enrichment

Enrichment Summary:

Entity Linking Coverage. Out of the 119 named entities extracted, 100 were successfully enriched via DBpedia queries, achieving a linking rate of 84%.

2.3 Embedding Structure Analysis

We then analyzed the structure by extracting the embedding matrices of entities and relations for each model. Their dimensions were printed to inspect the latent space size.

RDF triples were converted into PyKEEN format (TSV: subject, predicate, object), then into a NumPy array to create a `TriplesFactory`—a core PyKEEN object enabling ID-based processing and clean splitting for training, validation, and test sets.

2.4 Model Training and Evaluation: Link Prediction

We trained three knowledge graph embedding models using PyKEEN:

- **DistMult**: a bilinear model using dot-product scoring
- **RotatE**: a complex-valued model encoding relations as rotations in complex space
- **TransE**: a translational distance model

The graph was split into 80% training, 10% validation, and 10% testing sets. Standard metrics were used to evaluate model performance:

- **Mean Rank (MR)**: average position of the correct entity in predicted rankings (lower is better)
- **Mean Reciprocal Rank (MRR)**: average inverse rank, emphasizing top-ranked predictions
- **Hits@k**: proportion of correct entities ranked in the top- k predictions

Hyperparameters such as embedding dimension and learning rate were tuned iteratively. Early stopping was applied based on MRR improvements on the validation set to avoid overfitting. For RotatE,

complex-valued embeddings were flattened into real vectors by concatenating real and imaginary parts for visualization purposes (PCA and t-SNE).

2.5 Initial Results and Fine-Tuning

Initial model performance showed moderate link prediction capability, with Hits@10 around 0.3 and MRR below 0.4. To improve this, we experimented with:

- Increasing training epochs
- Tuning the embedding dimension and batch size
- Testing different negative sampling strategies

These adjustments led to substantial improvements in ranking accuracy, particularly for DistMult.

2.6 Comparative Analysis of Embedding Models

We compared the final performance of all three models using the same embedding size (`embedding_dim` = 50). Each model learned to score triples of the form $(head, relation, tail)$ by minimizing a loss over true and corrupted examples.

Metric	DistMult	RotatE	TransE
Mean Rank (MR)	83.06	152.75	97.64
Mean Reciprocal Rank (MRR)	0.4702	0.3518	0.2157
Hits@1	0.4091	0.2822	0.0379
Hits@3	0.4962	0.3845	0.3561
Hits@10	0.5341	0.4299	0.4545

Model Comparison.

- **DistMult** consistently outperformed the others across all metrics, with a strong MRR and Hits@1 above 40%. Its bilinear structure appears well-suited to our enriched graph.
- **RotatE**, while more expressive in theory, showed poorer ranking performance. Its higher Mean Rank suggests difficulty placing correct entities at the top.
- **TransE**, the simplest model, achieved decent Hits@10 but very low Hits@1, reflecting limitations in capturing the complexity of multi-relational knowledge graphs.

In our case, DistMult provided the best trade-off between expressiveness and ranking accuracy. The quality of its learned embeddings was further confirmed through projection into 2D space using PCA and t-SNE (see Section 3).

3 Visualizations and Examples

3.1 Entity Embedding Projections (PCA and t-SNE)

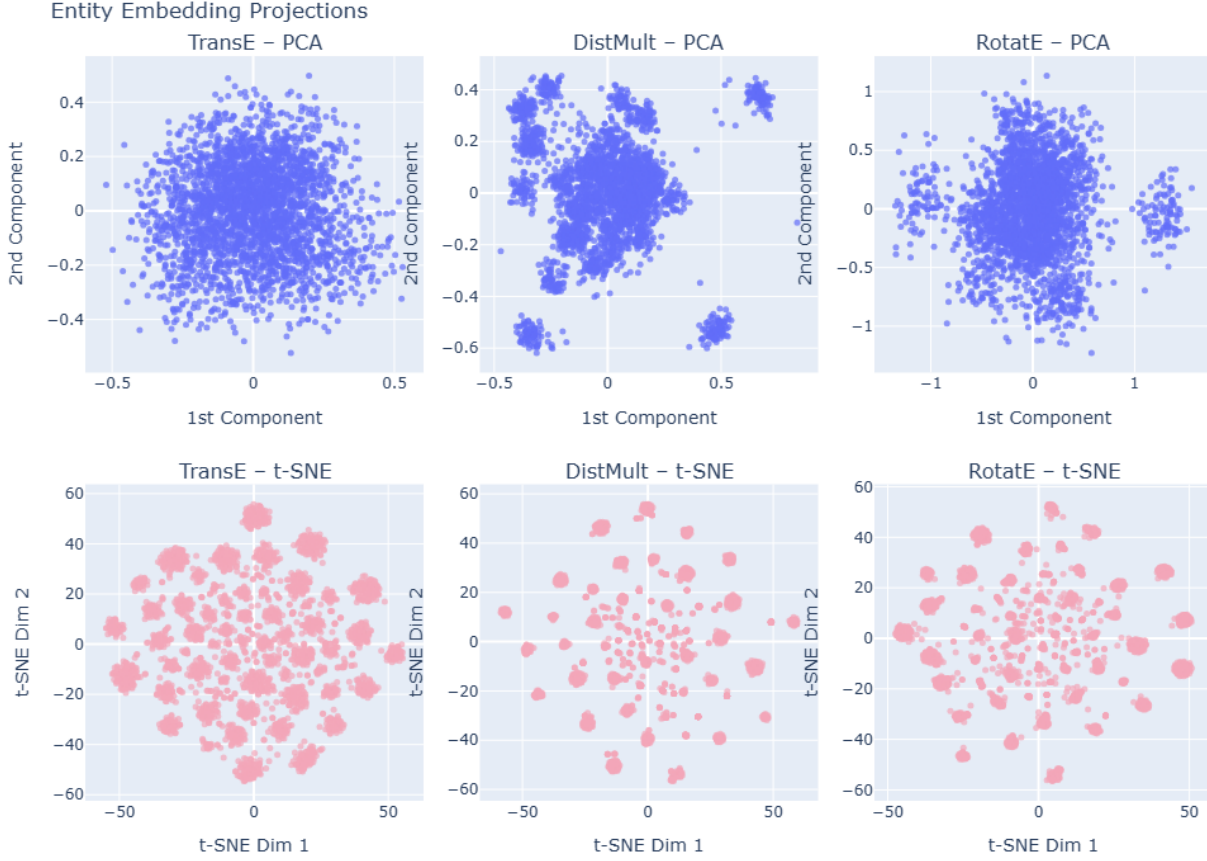


Figure 3: PCA (top) and t-SNE (bottom) projections of entity embeddings for TransE, DistMult, and RotatE

The figure above compares the spatial organization of entity embeddings for all models. Embeddings were projected from R^{50} (or C^{50} for RotatE, flattened into R^{100}) into 2D using PCA and t-SNE. These visualizations provide qualitative insights into how each model encodes semantic and relational structure.

- **TransE:** Displays a uniform cloud of points, lacking tight clusters or semantic separation. This flat structure reflects its limited ability to model complex, multi-relational graphs — consistent with its low Hits@1 and MRR.
- **DistMult:** Exhibits well-defined clusters in both PCA and t-SNE, revealing meaningful grouping by semantic roles (e.g., organizations, locations). This confirms the model’s high predictive performance and ability to differentiate entity types.
- **RotatE:** Shows moderate structure — more spread in PCA, and loose clusters in t-SNE. It captures local patterns (via complex rotations), but struggles with global alignment, reflecting its intermediate scores.

Summary: DistMult learns the most coherent and structured entity space. RotatE captures localized relationships with moderate success. TransE remains diffuse and less interpretable. These spatial patterns visually align with the ranking performance of each model.

3.2 Relation Embedding Projections (PCA and t-SNE)

We analyzed the spatial organization of learned relation embeddings across models using PCA and t-SNE.

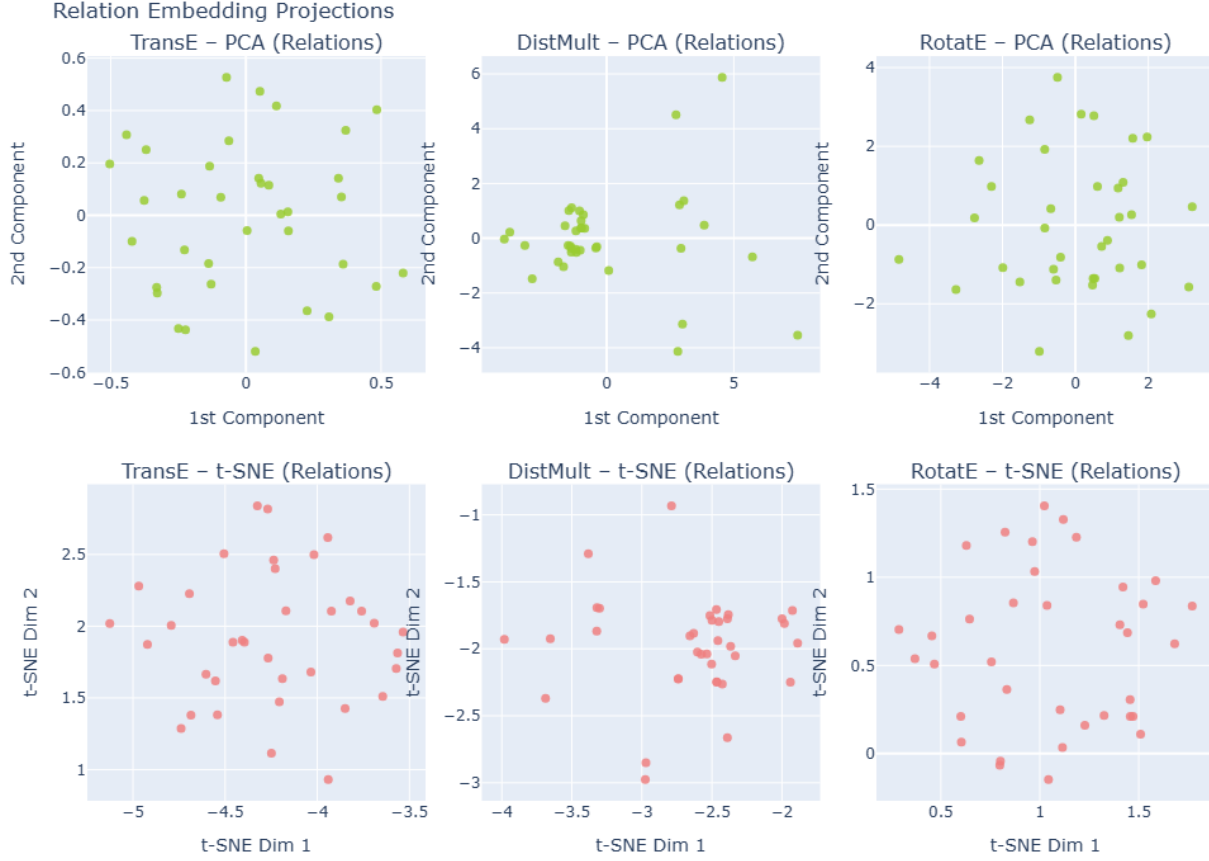
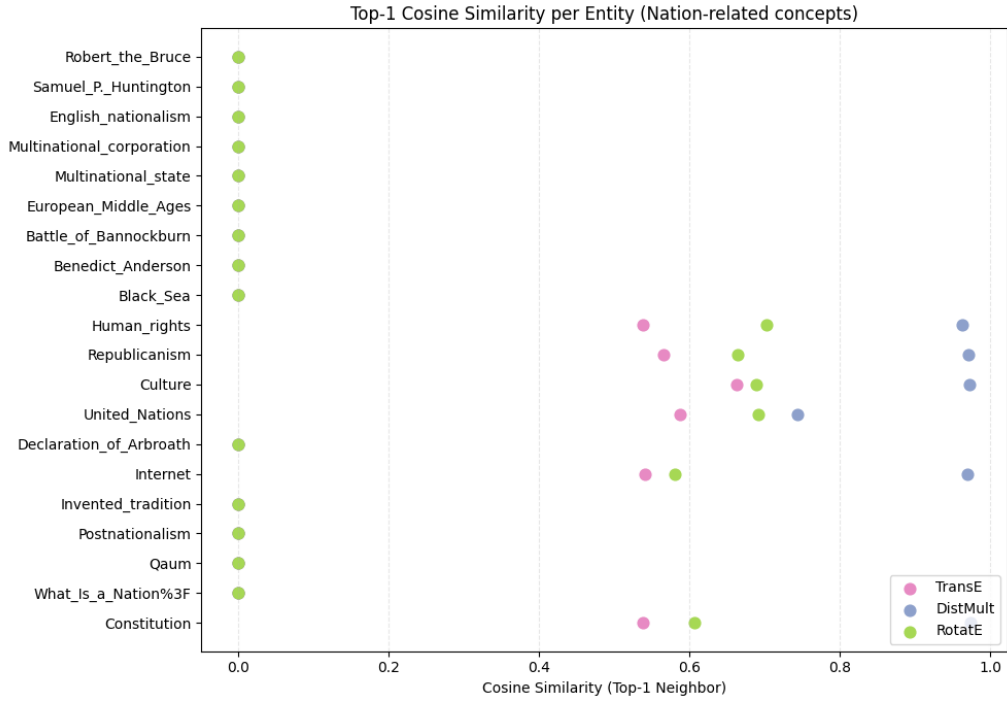


Figure 4: PCA (top) and t-SNE (bottom) projections of relation embeddings across all models

- **TransE:** Relation vectors appear uniformly scattered, with no visible grouping. This reflects the model’s simplicity and its limitation in capturing complex relational hierarchies.
- **DistMult:** Clearer grouping patterns emerge. PCA shows clusters near the origin and some spread apart, suggesting different functional roles. The t-SNE plot further confirms DistMult’s ability to encode and separate relation types effectively—aligned with its strong link prediction scores.
- **RotatE:** Exhibits more diversity in PCA, but less clarity in t-SNE. While it captures some asymmetries and cycles, the embeddings are harder to interpret without further regularization.



t-SNE projection of selected entity embeddings grouped by Top-1 neighbor similarity

This figure highlights clustering based on cosine similarity between each entity and its closest semantic neighbor. DistMult produces tighter and more interpretable clusters, RotatE shows moderate grouping, and TransE remains scattered. These distributions mirror model performance in both quantitative and qualitative evaluations.

Top 5 tail predictions for (Nation, wikiPageWikiLink, ?) using TransE:

<http://dbpedia.org/resource/Nation> – score: -5.8239
<http://dbpedia.org/resource/Republic> – score: -6.6009
<http://dbpedia.org/resource/Pennsylvania> – score: -7.0382
<http://dbpedia.org/resource/Latin> – score: -7.1195
http://dbpedia.org/resource/Republic_of_Ireland – score: -7.1384

Top 5 tail predictions for (Nation, wikiPageWikiLink, ?) using DistMult:

<http://example.org/people> – score: 1.0801
<http://example.org/smelling> – score: 0.5421
<http://example.org/tracked> – score: 0.4586
<http://example.org/politicians> – score: 0.4209
<http://example.org/Investors> – score: 0.4020

Top 5 tail predictions for (Nation, wikiPageWikiLink, ?) using RotatE:

http://dbpedia.org/resource/Chief_of_staff – score: -2.5355
<http://dbpedia.org/resource/Copyright> – score: -2.5839
<http://example.org/one> – score: -2.5880
http://dbpedia.org/resource/Substantial_form – score: -2.6191
http://dbpedia.org/resource/Intellectual_property – score: -2.6221

Figure 5: Raw scores from the `predict_tail` function for a given triple

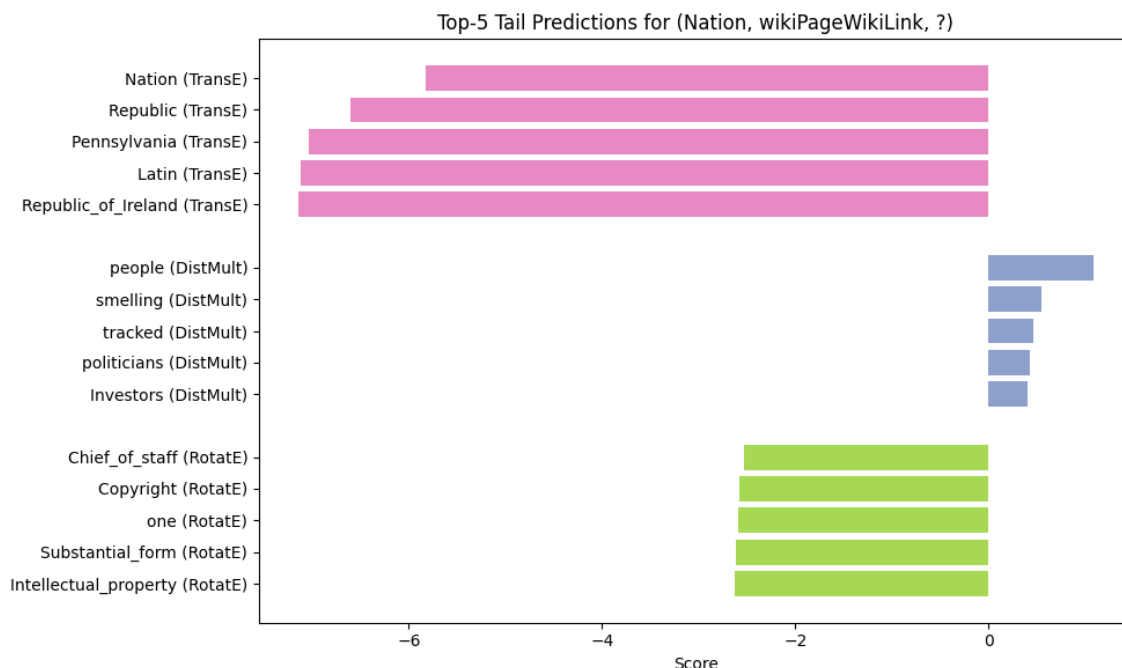


Figure 6: Top-5 predicted tail entities for (Nation, wikiPageWikiLink, ?) across all models

Interpretation. This barplot compares the top-5 predicted tail entities for a given head-relation pair. Scores reflect each model’s confidence.

- **TransE** returned geopolitically relevant terms like **Republic** and **Republic_of_Ireland**, reflecting its translational capacity to connect directly linked concepts.
- **DistMult**, despite its better ranking metrics, suggested overly generic or frequent entities like **smelling** or **tracked**, potentially indicating bias toward high-frequency terms.
- **RotatE** predicted more abstract entities like **Substantial_form** and **Intellectual_property**, which may suggest deeper semantic inference but also reduced interpretability.

3.3 Insights.

- Same-type entities often clustered closely
- Prominent people tended to appear near organizations they were linked to
- Outliers helped identify surface form collisions or enrichment inconsistencies

These visualizations confirm that enriched embeddings capture both semantic similarity and structural relationships. They also support downstream applications like entity clustering, recommendation, or automated graph repair.

Future work will focus on multi-model overlays, automatic cluster detection, and embedding trajectory analysis during training.

4 References Used

- StackOverflow: scraping, cookie handling : <https://stackoverflow.com>
- Official Documentation: Selenium and PyKEEN :<https://www.selenium.dev/documentation/webdriver/> + <https://selenium-python.readthedocs.io/waits.html>

- scikit-learn: for t-SNE : <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>

5 Conclusion

This project challenged us across multiple dimensions — from extracting structured knowledge from unstructured news articles to aligning it with global ontologies and exploring semantic embeddings.

Our end-to-end pipeline, from web scraping to link prediction, demonstrated both the complexity and the potential of web-based knowledge modeling. It strengthened our skills in information extraction, semantic representation, and model evaluation.

Future directions include:

- Enhancing entity disambiguation through contextual reasoning,
- Fine-tuning the NER pipeline with transfer learning techniques,
- Applying embeddings to clustering and recommendation use cases.